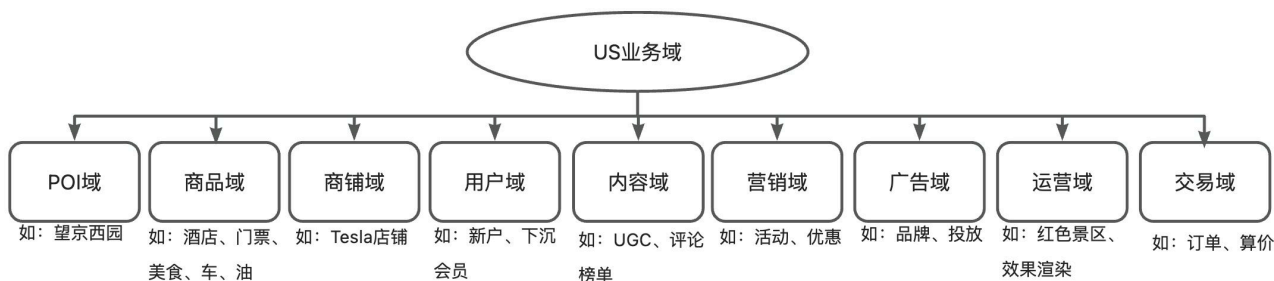


业务平台建设

1. 领域划分



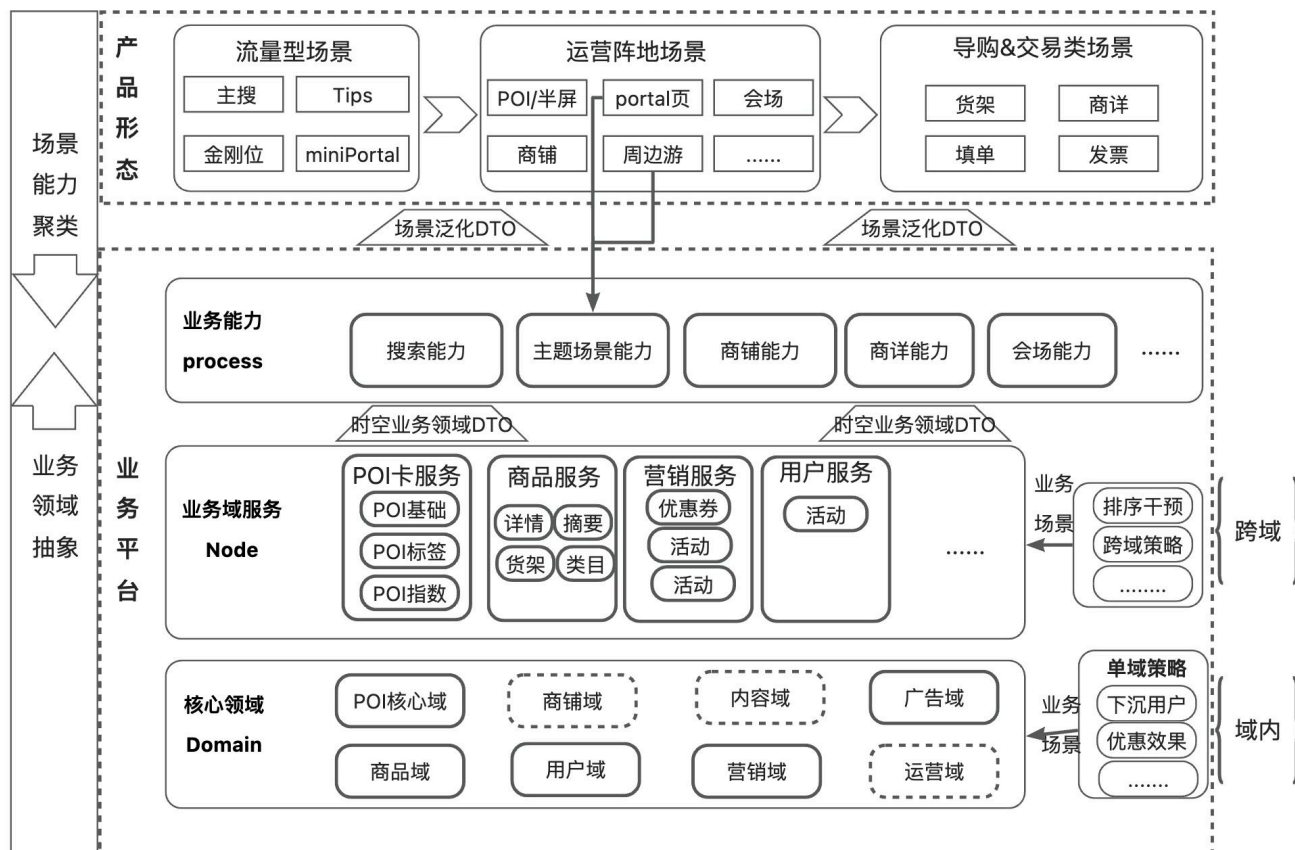
- 目前US在各个域内的逻辑相对复杂度不高，为**避免过度设计**，按照指导原则进行聚合设计出来8个领域；
- 支持在后续领域复杂度提升支持进行域的拆分；

参考域建设指导原则：[US业务域梳理及指导原则](#)

2. 业务架构

在特定场景（时间+空间）下，实现人与货的匹配，实现在高德体系下的用户LTV。

郭老师指导：在郭老师看来US就是数据+策略，标准能力建设上“主题场景能力”就应该把portal、miniportal、会场、周边游这些涵盖，它们之间没有差异，无非就是热搜词（金刚位）、feed推荐、广告或营销位；要有标准的输入和输出。思考：US是数据+策略：对于数据的理解，就是对基础服务平台的数据展示；对策略的理解，上百个字段到底要展示什么数据实现用户价值传递，行业是策略（portal上美食关注好评数、酒店关注商圈和品牌，房产使用POI标签，酒店使用商品标签）、场景是策略（中间页，下沉用户）；标准输入与输出：[核心域]->[时间，POI，商品/商铺/内容]



核心领域：对下层基础平台（商品服务平台，商家服务平台、广告平台、营销平台等）服务的调用，

解决了：

- 1) 为什么US核心领域和下游不一一对应？下游子域设计细节导致US业务平台的领域过度设计；
- 2) 在工程落地上承担了对下游服务非标准化服务的隔离（adapter）；
- 3) 提供领域内的扩展能力，形成规范的编码习惯；

- Input：下层平台服务输出的DO
- output：一套关于特定领域的输出；

业务域服务：结合 下层平台服务能力 和 时空场景下用户业务展开 的领域服务的抽象；

- output：由 基础平台服务的DO 和 用户产品策略DO 组成对 **时空** 业务的表达，三个维度的DO[核心域DO、POI域DO（空间）、用户产品策略DO（时间）]；

时空理解：用户在什么时间要什么商品或服务，购票、预约；下沉用户

场景编排：围绕 业务场景主题 的用户产品服务的表达

- output：由 多个业务域服务（如搜索结果页面：搜索框、筛选项、内容域榜单、POI列表等等组成）组成；

工程实践原则：

- 1) 统一原则

- 同级之间不允许相互调用，只允许上下层之间调用，业务及场景创新通过开闭原则中设计的扩展点实现支持；
- 扩展点管控原则：扩展点是基于业务来，每个扩展点有明确的目标，会基于业务流程模型设计需要开发，不会基于技术编码实现需要开放；

2) 搜推的特殊说明

- ComplexNode设计依据多品类召回的能力设计，满足设计原则中的迪米特法则（最小场景和能力聚合）和开闭原则实现，ComplexNode可以调用Node服务；为了保持统一的研发标准和高效的工程效率，其他场景ComplexNode原则上不开放，一事一议。
- searchProcess可被process调用，基于搜索场景分析 搜索流程偏对搜索动作控制和策略设计（与业务类流程的设计有差别），单独做特性支持；目前针对US已知场景业务，没有发现有对子process的使用诉求。

参考（portal及周边游设计）：

[portal页基于领域的详细设计](#)

[portal页研发成本评估](#)

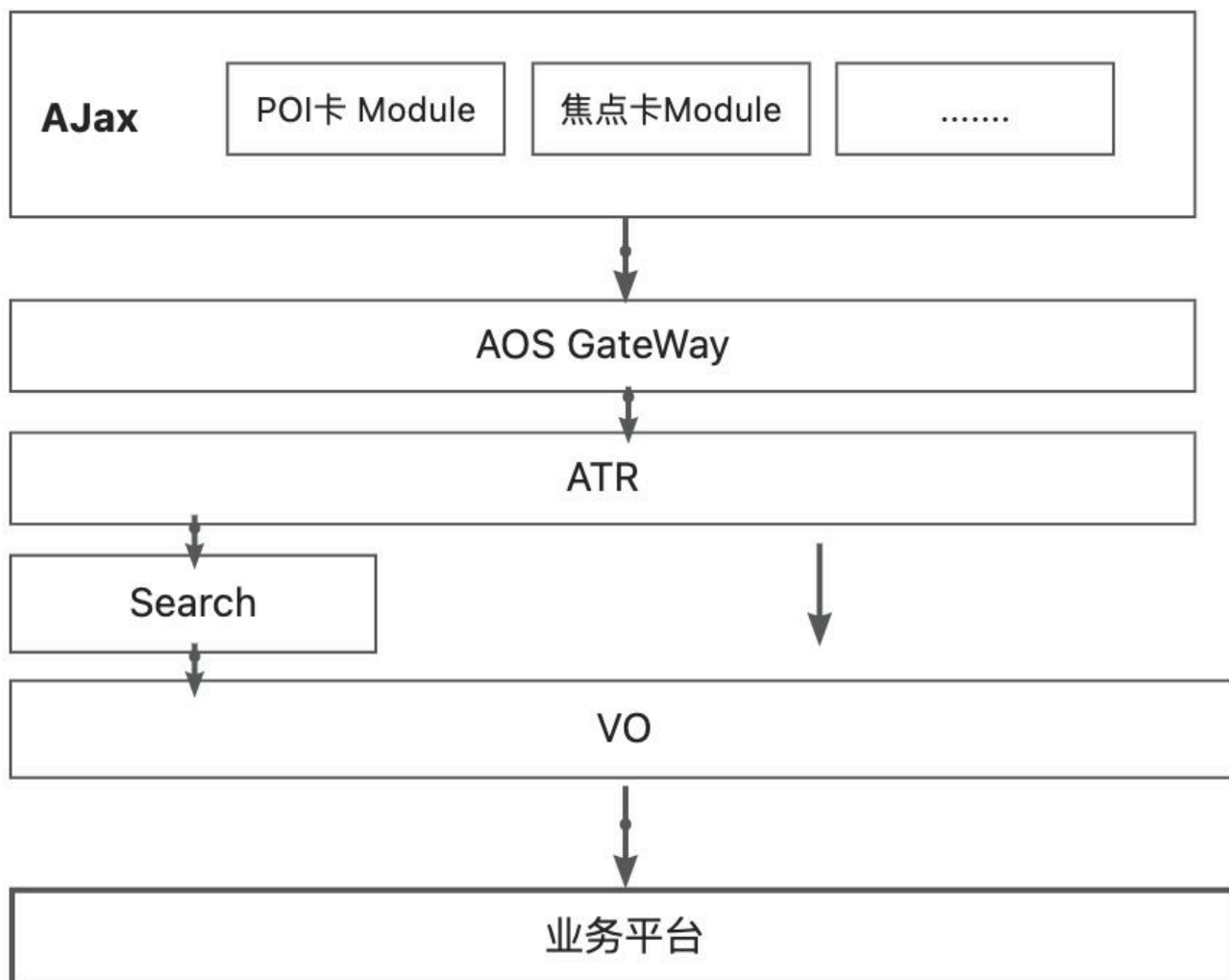
[POI基础领域模型](#)

[商品领域模型](#)

[内容模型](#)

3.服务调用链路

酒旅业务平台考虑 单体部署 模式；



未来可依据域服务的复杂度、服务分级治理、稳定性需要进行拆分，主要工作及成本：

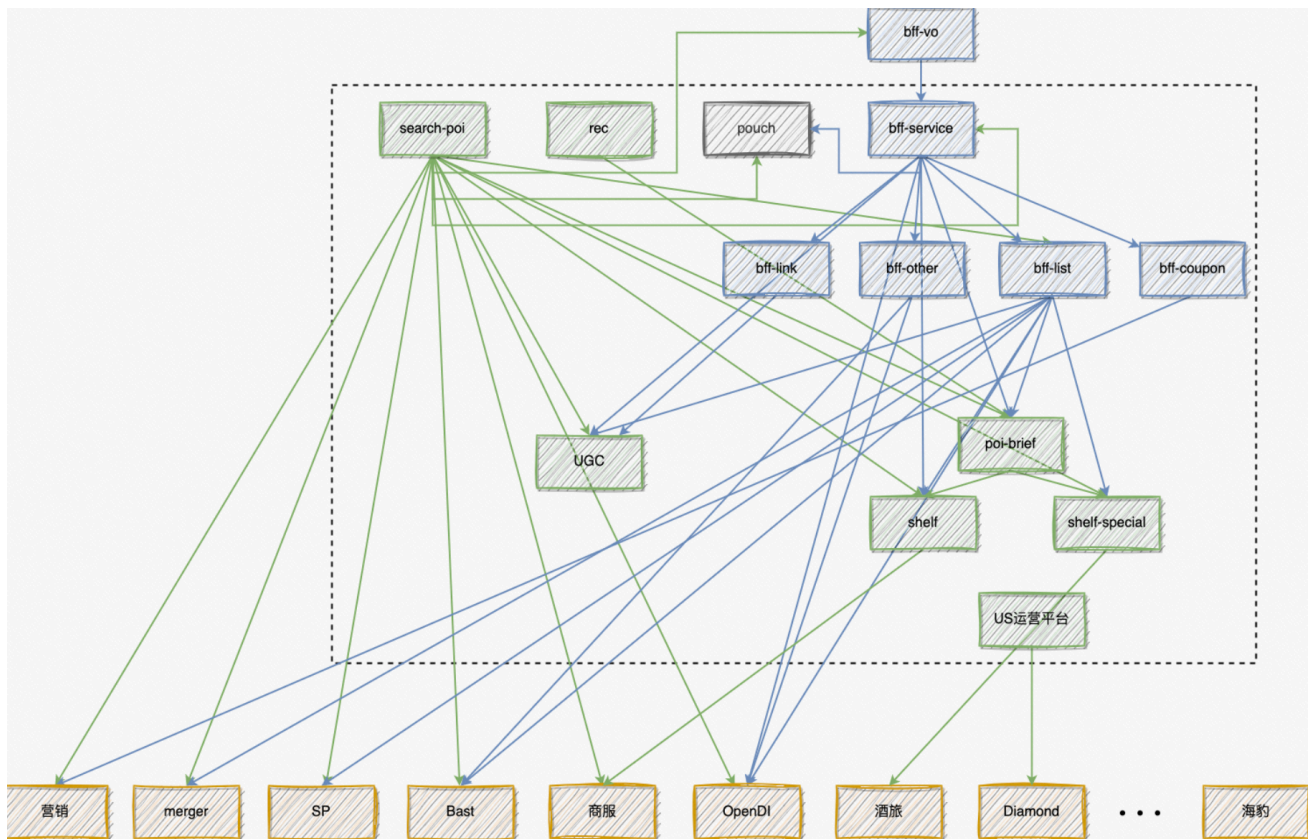
- 1) 拆包，将需要独立的服务单独打包；
- 2) 本地调用转RPC，可以依赖hsf的RPC特性，将本地调用转化成RPC调用；

成本评估：目前评估不涉及业务逻辑修改；

4.业务平台专项拆解及节奏说明

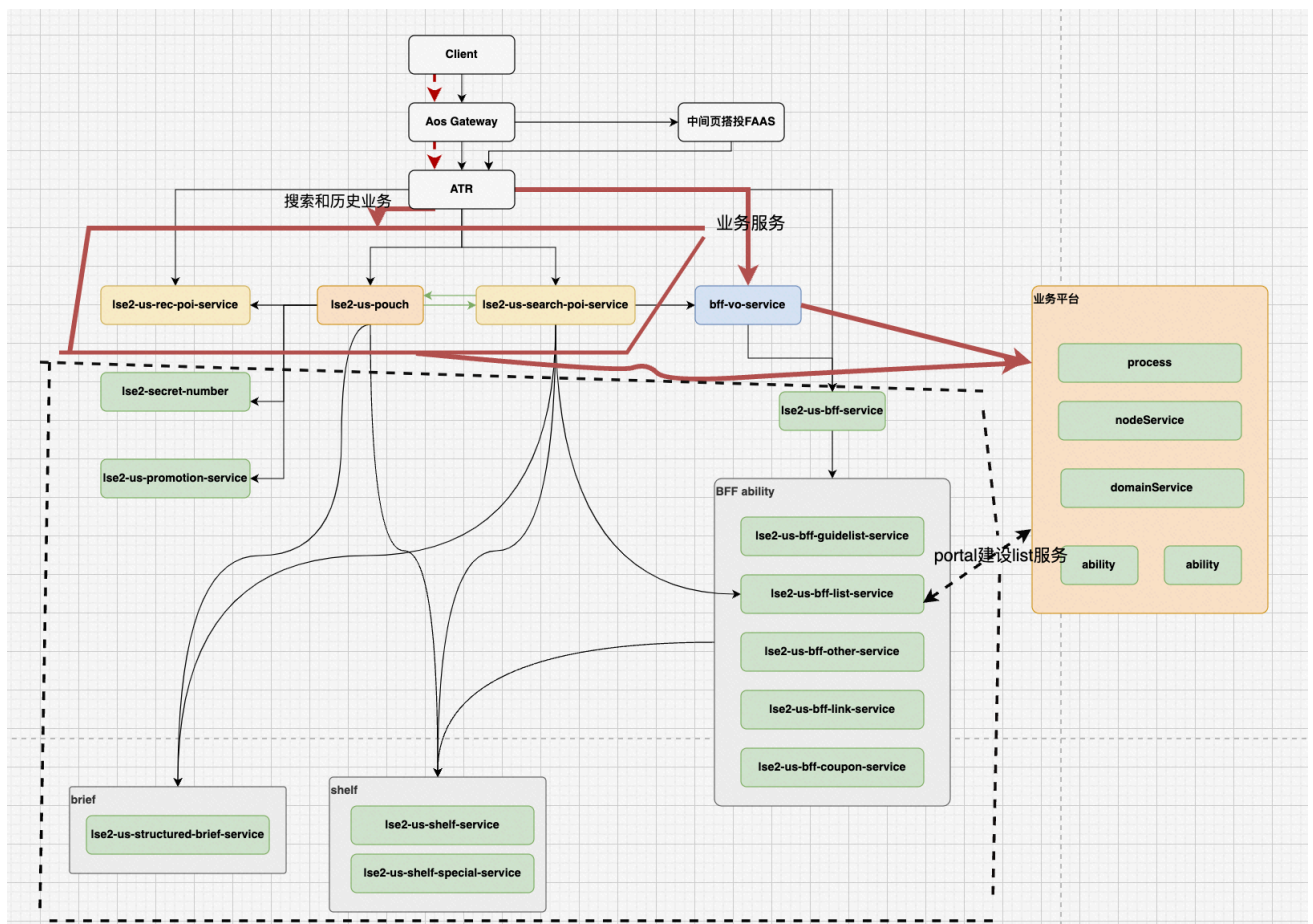
4.1系统中间态和终态分析（简版示意图）

US系统、子系统之间调用现状



目前US调用链路较深，最长链路6层(Search->VO->service->list->brief->shelf)

系统终态示意图



关键问题说明：

- 老的java服务通过迁移完成下线
- php持续下沉到新系统
- 由于老版本支持需要，部分老系统只做维护不做迭代

4.2 任务拆解

• 任务拆解原则

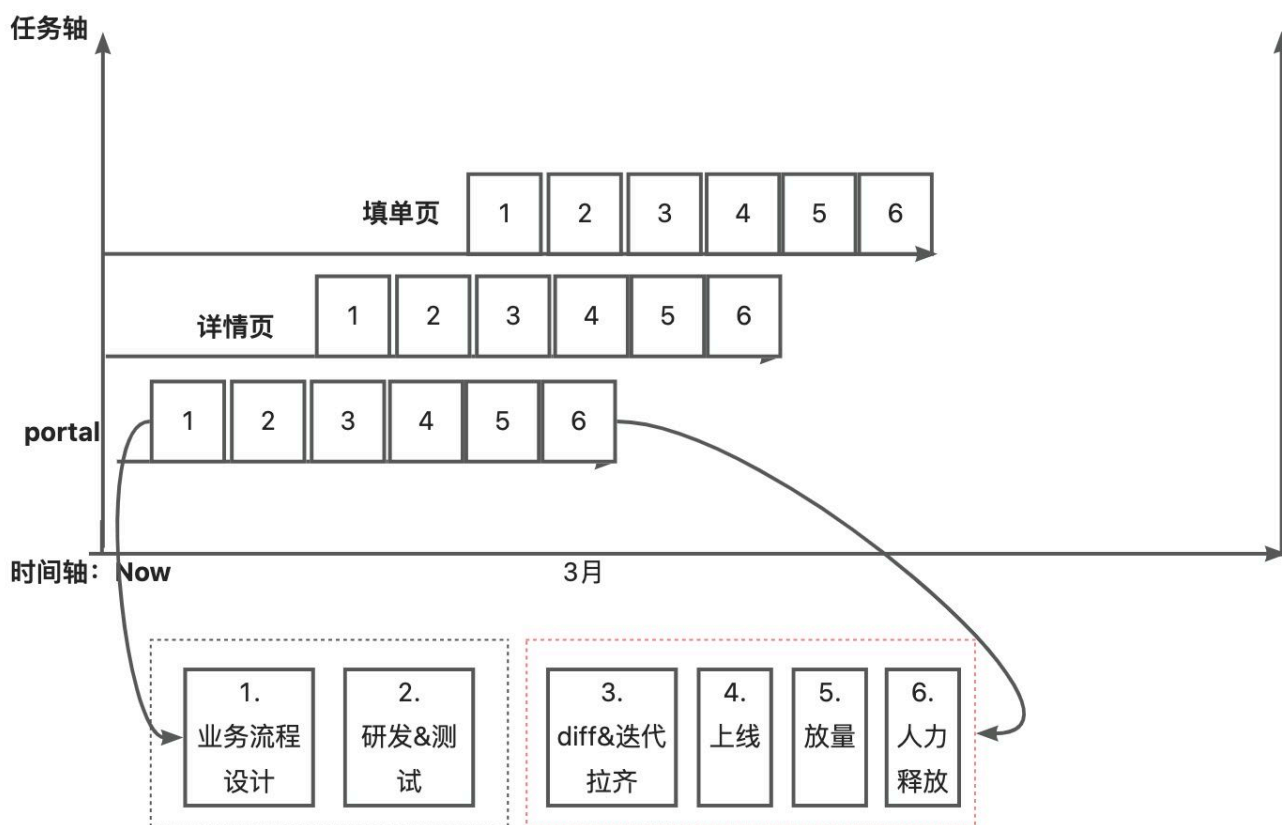
1) 大节奏：不阻断业务迭代，新、老系统分别开发部署，通过diff明确迁移可迁移状态，按评估节奏进行切换；（参考PHP向JAVA迁移）

说明为什么新方案对老系统的开发成本低（领域能力的复用将开发工作内聚到领域内，平台开发和业务开发分离解决了研发相互依赖）

2) 对US现有基础模块迁移：通过域能力建设完成对现有模块（list，brief等）的迁移和替换

3) 对前端流量入口迁移：按照场景分批次建设和迁移（如portal页，周边游）；

4) 研发及迁移节奏说明



说明：随着前端产品的迁移，后端的领域服务能力建设逐步验证和增强，研发成本逐步降低；

- 186个接口的梳理场景（带流量说明，帮助识别后续迁移的优先级）

附：[US服务接口](#)

流量型场景：搜索、金岗位等流量入口

运营阵地场景：portal页、POI页/半屏态、落地页、会场等

导购&交易场景：

能力建设项	对应用户产品	当前进展	排期	备注
主题场景能力	portal页（不含房产） 周边游	开发进行中	业务流程设计 开发完成 + 3.5完成 日常迭代和diff 维护 流量切换 上线完成	

4.3落地节奏

背景同步：

- portal页完成（95%）酒旅已具备可联调和后续规划上线的能力；通过portal页统一，可以完成多个核心模块的方案按照领域模型统一实现；
- 填单页完成方案梳理，从领域设计的层面看可统一酒旅填单页，初步判断美食、生服、黑卡可复用统一；[填单页梳理](#)
- POI详情页调研，两个结论：



复制代码

1) GBF可以支持POI详情的开发，

2) 开发收益单从POI详情页不具备可量化参考指标，应该从团队协作维度来看整体收益，
通过统一领域建设实现方案统一和提效;

- 三大战役：ACC增长战役（权益+优惠 全域分发：分群、价格、服务，爆品：爆品会场，流量土地：中间页）、主搜链路结构化战役（列表页、POI三态）、目的地战役（完成城市页、附近页、商圈、POI的产品迭代）

节奏规划：

- 通过三大战役，将城市页、附近页、商圈、会场、中间页、POI等产品方案收敛统一到业务平台；
- portal+填单由技术推动方案的一致，暂定portal在5月中旬完成上线、填单在6月底前完成上线；

- 商品详情页 暂定作为 ACC增长战役的交易链路进行技改；
- 其他模块如无迭代需求，S1暂不做调整，统一在S2进行规划；

风险及解法：

- 1) 架构和技术方案统一：系统设计和调用链路 由技术明确，通过 **域服务** 收敛统一服务调用；
- 2) 产品形态和技术方案达不成一致的，明确具体的分歧点，反馈 帅哥和搜索负责人李刚老师 从上层推动解决；

原则：

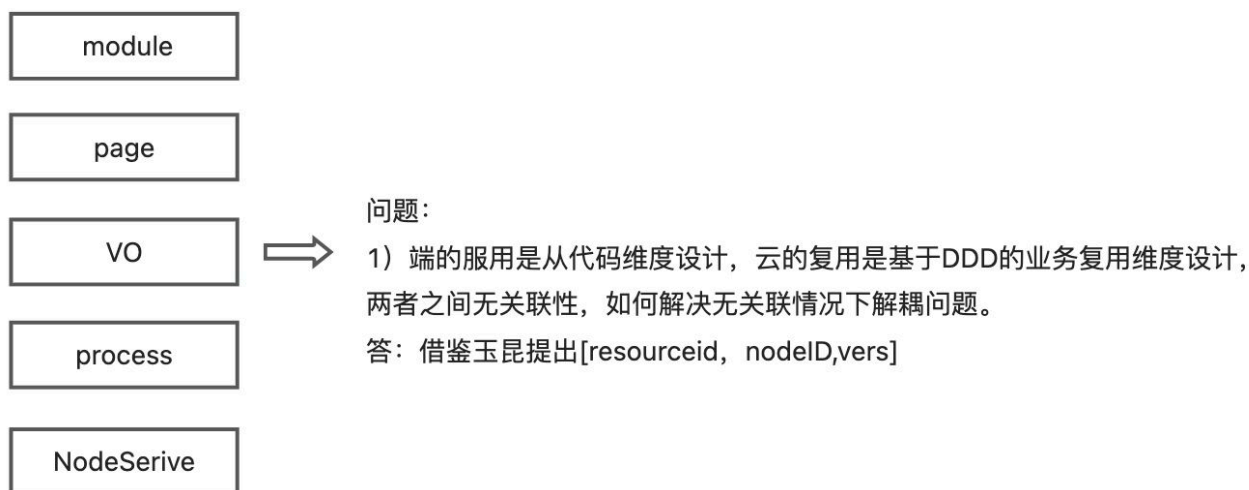
标准能力建设不是为了还原现在不合理的调用链路，而是通过统一的 领域建设 完成业务架构分层 和 统一技术架构，实现清晰合理的调用关系，提升工程效率。

4.4 资源&风险依赖

- 1) 行业的需求和功能支持下沉平台：业务流程设计过程，业务线有专门接口人对接，确认对当前业务需求的产品效果一致；
- 2) 合作团队的资源和节奏：前端VO开发节奏是否一致；测试资源如何保障
- 3) 业务支持迁移平台：业务线研发安排专人支持迁移（迁移的主负责人是业务线研发）
- 4) 业务需求落地平台需求的流程机制：平台上线完成后，业务产品需求如何转化成平台需求

5.其他问题备忘

5.1 与VO对接



目前端推荐使用**端云一体化方案**，潜在稳定性风险分析：

- 1) FaaS因业务需求多次调用后端
- 目前根据晓博和齐然前期的规范要求，只允许调用一次，这个风险可忽略

2) 云端切换的方案是否依赖发版，是否能人工强行干预

- 通过返回错误码端上自动切换，
- 依赖云控设计一个人工介入强制切换方案，VO服务异常端强制全部走端的方案，缺点是需要APP重启或主屏切换来生效

3) 对云服务的流量由于自动切换翻倍问题；

- 对端上的发送给VO的流量和不过VO的流量进行染色；
- 可以提前做好降级预案，在流量达到阈值后启动服务降级，将对VO的请求进行降级（兜底方案是什么？晓博反馈端有一个统一的兜底页面）
- 流量降级后，通过人工干预介入决定（切机房、切接口）

4) nodeJs代码更新时效

- CDN是否能全部被更新，什么机制保障
- 暂定20分钟（可调整），如果nodeJS代码出现bug，如何保证服务快速恢复，影响待确认

5) DTO的数据量过大问题

- 最大数据量是3M，影响待确认

6) 手机体验问题

- nodeJS代码被编译成Ajax代码在端上是否会耗能增大明显被手机安全助手杀掉。影响待确认